ChatGPT

Detailed Plan for Roadmap Step 3: Pilot Development Phase

Overview of Step 3 (6–10 Week Pilot Development)

Scope: In this phase we will implement a **pilot version** of the system with detailed features, following the roadmap's next planned enhancements [1] [2]. The focus is on **low-level, concrete tasks** – nothing is left ambiguous or "up to interpretation." All requirements will be broken down into epics, user stories, and acceptance criteria, making development straightforward and laying the groundwork for future testing.

Timeline: One developer (yourself) will execute Step 3 over ~6–10 weeks, which suggests roughly **3–5 two-week sprints**. We will prioritize core improvements first (e.g. form enhancements, data tracking), then move to mapping and analytics. Each epic below includes specific features and tasks with clear outcomes. Automated testing and multi-user support are **out-of-scope for this step** – however, we will design the system in a way that adding these later is seamless (e.g. documenting requirements for future test cases, keeping user roles in mind). By the end of Step 3, we expect a robust single-user pilot with well-documented features (epics, stories, functions) ready for user validation.

Epic 1: Fumigation Process Enhancements

Goal: Streamline and error-proof the fumigation scheduling workflow, improving user experience and data quality. *(Roadmap Phase 2 features)*

- **Multi-Step Fumigation Wizard:** Break the current single-page fumigation form into a logical sequence of steps (e.g. Step 1: Basic info, Step 2: Treatment details, Step 3: Geospatial selection, etc.). This wizard-style interface will simplify complex data entry by showing fewer fields per screen, reducing cognitive load and user mistakes [3] [4]. Each step may branch based on prior answers (e.g. if "Equipment" = **Drone**, show a step to upload flight plan). Acceptance Criteria: The user can navigate forward/back through all steps, with validation at each step, and see a progress indicator.

- **Auto-Save Drafts:** Implement automatic saving of form data as the user fills out each step (e.g. save to `localStorage` or an in-memory draft every few seconds or on each step completion). This prevents data loss if the app closes or the user navigates away [5] [6]. When the form wizard re-opens, it should detect any saved draft and offer to restore it. Acceptance Criteria: If the browser crashes mid-form, no more than one step of input is lost; users are notified of restored drafts upon returning. This provides a **smoother UX**, as users won't have to redo lengthy forms due to connectivity issues [7] [6].

- **Product Autocomplete Field:** Enhance the "Product" dropdown (e.g. list of insecticides, herbicides, etc.) with an autocomplete search feature. As the user types, show matching product names, allowing quicker selection. This saves time and reduces input error by leveraging "type-ahead" suggestions [8]. We will use a known library (or HTML datalist) for reliable results.

Acceptance Criteria: Typing at least 2 characters in the product field shows a suggestion list of matching products; selecting a suggestion fills the field.

- **Automatic Cost & Dosage Calculations:** Where possible, auto-calculate related values to aid the user. For example, if "Area treated" and "Dose per hectare" are provided, auto-calc total volume; or if "unit cost" of a product is known, auto-calc total cost for the treatment. This reduces manual math and ensures consistency (automated calculations help eliminate data entry errors) [9]. Acceptance Criteria: The form updates calculated fields in real-time once required inputs are provided, and users can override if needed.

- **Status Change Log (History Tracking):** Every fumigation record will maintain a history of status transitions (e.g. Planned → In Process → Completed, including timestamps). This is essentially an **audit trail** of changes. It will be recorded in the data model (e.g. an array of `{status, timestamp}` entries) whenever the status field is updated [1]. Though largely internal, this prepares for future multi-user accountability and provides context for each record's lifecycle. Acceptance Criteria: When a fumigation's status is changed, the old status and time of change are appended to its history; the UI "Details" view can display a simple timeline of statuses. (This will later help in building analytics like average time in each status.)

Epic 2: Advanced Field & Environment Management

Goal: Enrich the field data model with soil and climate information and introduce intelligent recommendations. *(Roadmap Phase 3 features)*

- **Soil Characteristic Tracking:** Extend the **Field (Lote)** entity to capture key soil attributes for each field. This includes fields like soil type/texture, pH, nutrient levels, organic matter %, etc. Provide UI in the Field form to input/edit these characteristics. The benefit is to support better agronomic decisions – as seen in farm apps that let users record field-specific soil data for planning [10]. Acceptance Criteria: Users can record soil properties for a field (at least soil type, pH, N-P-K levels); these are displayed on the field's detail view. *Example user story: "Como administrador de campo, quiero registrar las características del suelo de cada lote (pH, nutrientes) para tomar decisiones de fertilización informadas."* **Rationale:** Documenting soil data will help in choosing the right treatments – e.g. knowing a field's low potassium can trigger a fertilizer recommendation [10].

- **Climate Data Integration:** Integrate weather and climate information into the app to contextualize treatments. We will start by pulling **real-time weather data** (e.g. via a free API like OpenWeatherMap) based on the field location or user location. This can populate fields like current temperature, humidity, wind speed, etc., which are already part of the fumigation record's "condiciones climáticas." Additionally, allow viewing a **5-day forecast** when scheduling a fumigation (to pick optimal weather windows). This addresses the need for farmers to have weather data alongside their crop data – modern farm apps integrate forecasts and sensor data to help quick decisions [11]. Acceptance Criteria: On the fumigation form, the user can fetch current weather for the selected date/location (or it auto-fills); if wind speed exceeds a threshold, display a warning ("Caution: High wind – spraying might drift"). Ensure offline fallback (e.g. if no API data, user can manually enter conditions). *Example:* If planning a spray and forecast shows rain, the system could alert the user. *(Full IoT sensor integration is beyond this scope, but the architecture will allow plugging in sensor data later.)*

- **Field-Specific Treatment Recommendations:** Provide basic decision-support to the user by suggesting recommended actions or products for a field based on its data. This could start simple: e.g. *"Field A has acidic soil (pH 5.2); consider lime treatment before planting."* or *"High pest incidence last month – recommend a preventive fumigation."* We will create a rule-based engine or use lookup tables for a few scenarios. The idea is to move toward the "smart advisory" features noted in the roadmap [12]. Even a rudimentary system will lay groundwork for future AI models. Acceptance Criteria: When viewing a field or planning a fumigation, the system shows at least one relevant suggestion (with an explanation). For example, if soil organic matter is low, a note could say: "Consider cover cropping to improve soil health." **Note:** This mimics decision-support systems that guide farmers by analyzing data [13]. All recommendations will be documented as static rules for now (e.g. within code or a config file) – implementing a full ML-based engine is beyond 10 weeks but we will structure the code to allow plugging one in later.

- **Data Model & Design Considerations:** All new data (soil info, weather info, recommendations) will be stored client-side for now (since we have no backend in the pilot). We will ensure the data schema can later expand for multi-user. For instance, the Field data structure will include an owner/user ID (even if in pilot there's only one user) and we maintain the `user.role` field that already exists [14] [15]. This way, when multi-user and cloud storage are introduced, we can migrate local field records to the backend per user. We'll also keep climate and soil data modular (e.g. weather data fetched via a service module) so that swapping in a different source or backend later is straightforward.

Epic 3: Geospatial Tools & Visualization Enhancements

Goal: Improve the mapping interface with measurement tools and layered data visualizations. *(Roadmap Phase 3 feature continuation)*

- **Distance & Area Measurement Tool:** Add an interactive measuring tool on the map. Users should be able to click points on the map to measure distances (e.g. length of a fence line or spray run) and areas (arbitrary polygons). This is a common feature in farm mapping software, helping users estimate field extents, plan fences or irrigation layouts, etc. For example, AgriWebb's farm map offers a simple distance tool for planning and insurance claims [16]. We will integrate an existing Leaflet plugin (like **Leaflet.MeasureControl**) for this. Acceptance Criteria: A "Measure" button on the map opens the tool; the user can draw a line or shape and see the total distance (in meters) or area (in hectares) dynamically. They can exit the mode without affecting stored data. This tool does not save measurements (ephemeral use), but the user can screenshot or record values.

- **Multi-Layer Map Visualization:** Enable the map to overlay additional data layers such as soil maps or weather layers. In this pilot, we will implement the ability to toggle a **soil fertility heatmap layer** (if data available) and a **precipitation radar layer** from a public source. The idea is to display more insights visually – e.g. show soil type variation across fields, or current rainfall. We will use tile services or simple colored overlays for this. **Example:** The user clicks "Show Soil Map" and sees a color-coded layer indicating soil pH or type per area (using dummy data or an external WMS for demonstration). This aligns with industry trends where GIS layers (soil, satellite imagery, etc.) are integrated for precision farming [17]. Acceptance Criteria: At least two overlay layers are available on the map (soil and weather). They can be toggled on/off independently and are displayed with a legend. Performance should be acceptable (since this is client-side, we'll keep resolutions moderate). Documentation: We will note how to integrate additional layers in the future (e.g. NDVI vegetation index) using similar mechanisms.

- **UI/UX Improvements for Mapping:** Minor improvements such as color-coding fields by status (already partly implemented) and ensuring the map and drawing tools work on mobile screen sizes. We'll verify that the polygon drawing (field boundaries) and the new measure tool are touch-friendly. This ensures the pilot's geospatial features are **mobile-friendly**, given many field users rely on smartphones. We won't overhaul the entire UI, but we will make incremental tweaks (like slightly larger hit areas for map buttons, etc., guided by usability best practices).

Epic 4: Advanced Analytics & Reporting

Goal: Augment the application's analytics dashboard with real-time metrics, trends, and reporting capabilities. *(Roadmap Phase 4 features)*

- **Real-Time KPI Dashboard:** Evolve the existing dashboard to update metrics in real-time as data changes, and introduce a few new key performance indicators. For example, display **"Fumigations Planned this Month" vs "Completed"**, **Total hectares treated this year**, etc., in stat cards. We'll include charts that update when underlying data is added (since we're in a single-page app with no backend, "real-time" means on each session or action). The roadmap explicitly calls for a KPI dashboard [18] – implementing this will give users immediate insight into their operations. Acceptance Criteria: The dashboard screen shows at least 4 KPI cards (e.g. # of fumigations this month, % of fields with updated soil data, etc.) and 1–2 charts (e.g. a bar chart of fumigations per month). When the user adds or edits records, the numbers update accordingly. This supports faster decision-making (highlighting issues like under-treated fields) [19].

- **Temporal Trend Analysis Charts:** Add analytics views that illustrate data over time. For instance, a trend line of the number of fumigations per month throughout the year, or an area chart of cumulative hectares treated over time. Another valuable analysis is effectiveness over time: if we capture any outcome metrics (e.g. pest population before/after treatment as a proxy for effectiveness), we could graph that trend. These trends help identify patterns and measure progress (e.g. are we fumigating more in spring than fall? Is treatment effectiveness improving?). Acceptance Criteria: At least one time-series chart is available (e.g. a line chart of monthly fumigation count). The chart can be filtered by year or by field. Users can derive insights such as seasonality of activities. *(If actual effectiveness data is not available in pilot, we will simulate or use "completed vs cancelled treatments" as a measure of success rate over time.)*

- **Predictive Outcome Indicator (Prototype):** While true predictive modeling is likely beyond the pilot timeframe, we will include a placeholder for "Effectiveness predictive models" [18]. Concretely, we might implement a simple rule-of-thumb predictor – e.g. if a field had >3 fumigations in a year, predict a higher pest pressure next year (just as a demonstration). This will be presented as a note on the dashboard like *"Next season pest pressure: HIGH (predicted)"*. The purpose is mostly to reserve a spot in the design for future ML integration. We will clearly label such output as experimental. Acceptance Criteria: One predictive insight is shown based on current data (even if simplistic). The logic behind it is documented, and the interface can accommodate a more complex model's output later.

- **Custom Report Export (Basic Version):** Lay the groundwork for a **custom report builder** [20] by allowing users to export certain data views. In the pilot, this could be as simple as a "Download CSV" or "Print PDF" for key tables (e.g. list of all fumigations, or field details with soil info). The UI will have a Reports section where the user can select a report type (Fumigation Log, Field Summary, etc.) and either view it formatted or export it. We will leverage existing UI components

for lists and perhaps an HTML-to-PDF library for export. Acceptance Criteria: The user can obtain at least two types of reports: (1) a **Fumigation Summary Report** (all records with key fields) and (2) a **Field Detail Report** (one per field including soil and activity summary). Even if rudimentary, these fulfill the need to share data externally (e.g. with an agronomist or for compliance). The code will be structured so that adding new report formats (or a more interactive builder) later is straightforward.

- **Mobile-Optimized Charts and Layout:** Ensure that the analytics dashboard and charts are responsive. Many farmers or agronomists might use tablets or phones, so our charts (built with Recharts) should resize or provide toggles for simpler views on small screens. We will test the dashboard on a mobile viewport: KPI cards should stack vertically if needed, and charts should scroll or condense. Acceptance Criteria: Viewing the dashboard on a 5-inch smartphone screen still shows all key info without broken layouts (this might mean using a simpler chart or hiding non-critical elements). This task aligns with best practices (ensuring data visualizations remain usable on mobile) and will improve user adoption of the pilot.

Epic 5: Documentation & Quality Assurance Preparation

Goal: Rigorously document every feature (epic, story, function) and set the stage for future test automation and team onboarding.*

- **User Story & Acceptance Criteria Documentation:** For each user story implemented in Epics 1–4, we will write a clear description and acceptance criteria in the project documentation (e.g. in a Notion page or markdown file). This ensures that nothing is left to guesswork and that testers (once involved) can derive test cases directly from these criteria [21] [22]. For example, the story "As a user, I want autosave so I don't lose form progress" will have acceptance criteria like "If the browser is closed mid-form, upon reopening, the form is restored to the last filled state." Well-defined acceptance criteria are extremely helpful for QA, providing a solid basis for test case writing [22]. We'll follow this practice for every feature to avoid any ambiguity (this addresses the common pitfall that without detailed documentation, some scenarios remain untested [21]).

- **Technical Documentation Updates:** We will update relevant sections of the architecture docs and README. For instance, document new localStorage keys introduced (e.g. draft autosave storage structure), any new major components (e.g. `WeatherService.js` for climate integration), and usage instructions for new features (perhaps an updated README section on "New Features in Pilot v2"). Maintaining internal docs is critical since other developers or stakeholders might join after the pilot. We'll also keep an ADR (Architecture Decision Record) for any notable decisions in Step 3 (e.g. "Chose OpenWeather API for weather integration due to simplicity – will revisit for enterprise source in Phase 5").

- **Code Quality and Testing Hooks:** While formal automated testing will be added later, we won't neglect quality in this stage. We will **refactor where needed** (ensuring code from MVP is clean and modular for new features) and add **instrumentation for testing**. This could include: enabling Jest in the project (if not already) and writing a **couple of sample test cases** as templates – e.g. a test for the autosave function (simulating form input and checking localStorage) – to demonstrate how to test our new features. We'll also configure linting (ESLint) properly to enforce coding standards (the analysis noted a missing `.eslintrc` [23]). These steps ensure when we do add a full test suite, we have a head start.

- **Manual Testing & UAT:** As the sole developer, you will perform thorough manual testing for each user story upon completion (e.g. following the acceptance criteria like a checklist). We will document any edge cases discovered and how they were resolved. Additionally, if possible, a friendly user (perhaps the agronomist who gave requirements) can do a **pilot user acceptance test** of the new features at the end of the phase. Their feedback and any detected issues will be recorded. This not only helps fix issues now but also produces documentation for future testers. Each story's documentation will effectively double as a test case (the acceptance criteria list) which QA can later convert into formal test scripts [21].

- **Preparation for Multi-User & Roles:** We keep in mind that after the pilot, the system will evolve to support multiple users and roles (Phase 5 of the roadmap [24]). To not "paint ourselves into a corner," we will document assumptions and necessary changes for that future step. For example, note that currently all data is global, but in multi-user mode, fumigations will belong to a user or org – so our code should be reviewed for any hard-coded user references. Similarly, any new feature that would behave differently with roles (e.g. an "Operator" might not see cost analytics) will be flagged in comments or docs. By acknowledging these now, we ensure Step 3's work does not create obstacles for the role-based multi-user expansion [15] [24].

Each of these epics and tasks will be executed with a **"definition of done"** that includes updated documentation and verification against acceptance criteria. By the end of the ~8-week period, we will have a feature-rich pilot application covering enhanced forms, data tracking, geospatial tools, and analytics – all thoroughly specified. This detailed approach not only delivers a robust pilot now, but also makes future steps (adding more users, writing test cases, integrating a backend) far easier, since the groundwork in documentation and design will have been laid [25] [21].

Sources:

- Project Roadmap & Phases – *Development Plan (Phases 2–5)* [1] [2] [24]
- Nielsen Norman Group – *Benefits of Multi-Step Form Wizards* [3] [4]
- UX StackExchange – *Importance of Autosave for Long Forms* [5] [6]
- Coveo UX Best Practices – *Autocomplete Saves User Time* [8]
- PickApp Farm Software – *Cost-per-Hectare Calculations & KPI Dashboards* [9] [26]
- xFarm Features – *Soil Characteristic Tracking Benefits* [10]
- Glance Agritech Guide – *Integrating Weather Data for Farmers* [11]
- Senapaty et al. 2024 – *Decision Support Recommendations in Agriculture* [13]
- AgriWebb Blog – *Mapping Tool for Measuring Farm Distances* [16]
- Doktar AgTech Blog – *GIS & Soil Maps for Precision Farming* [17]
- TestLodge Blog – *User Stories, Acceptance Criteria and Testing* [25] [21]
- **(Additional context from FumigApp Technical Analysis & docs)**

---

[1] [2] [12] [14] [15] [18] [20] [23] [24] TECHNICAL_ANALYSIS_REPORT.md
file://file_00000000553c71f5aae7adfdf038de77

[3] [4] Wizards: Definition and Design Recommendations - NN/G
https://www.nngroup.com/articles/wizards/

[5] [6] [7] autosave - Form saving automatically - User Experience Stack Exchange
https://ux.stackexchange.com/questions/85619/form-saving-automatically

8 6 UX Design Best Practices for Autocomplete Suggestions
https://www.coveo.com/blog/autocomplete-suggestions-ux-best-practices/

9 19 26 What Is Farm Management Software? Features & ROI for 2025 - PickApp
https://www.pickapp.farm/docs/what-is-farm-management-software-features-roi-for-2025/

10 Managing Your Farm Fields via App | xFarm
https://www.xfarm.ag/en/field-management

11 How Do I Handle Weather Data And Crop Monitoring In My Farming App?
https://thisisglance.com/learning-centre/how-do-i-handle-weather-data-and-crop-monitoring-in-my-farming-app

13 A Decision Support System for Crop Recommendation Using Machine Learning Classification Algorithms
https://www.mdpi.com/2077-0472/14/8/1256

16 New mapping tool feature made to measure - AgriWebb
https://www.agriwebb.com/blog/new-map-feature-made-to-measure/

17 www.doktar.com
https://www.doktar.com/en/blog/digital-agtech/using-soil-maps-to-identify-field-fertility-and-nutrient-needs/

21 22 25 Writing Test Cases from User Stories & Acceptance Criteria - TestLodge Blog
https://blog.testlodge.com/writing-test-cases-from-user-stories-acceptance-criteria/